SEASUG Paper 109-2026

From SAS® to R: Five Essential Clinical Graphics Every SAS Programmer Should Know

George Wu, Jaime Yan, and Jessie Yan, Kardigan

ABSTRACT

Clinical reporting programmers have spent years mastering ODS Graphics — the SG procedures and the Graph Template Language — to build the figures reviewers expect: waterfall, swimmer, Kaplan-Meier, forest, and spider plots. As open-source tooling becomes a first-class citizen in clinical study reporting, the same programmers are increasingly asked to reproduce those figures in R. The good news is that most of what you already know transfers directly; only the vocabulary changes.

This paper is a practical, side-by-side field guide for the SAS programmer moving into R. Starting from a familiar `PROC SGPLOT` mental model, it introduces the grammar of graphics used by `ggplot2` and maps each SAS concept — plot statements, `GROUP=`, `REFLINE`, `PANELBY`, attribute maps, axis tables — onto its `ggplot2` equivalent. It then walks through five essential clinical graphics, showing working SAS and R code for each, every example built from the same simulated CDISC ADaM-style datasets (ADSL, ADTR, ADRS, ADTTE), along with the packages that do the heavy lifting (`ggplot2`, `survival`, `ggsurvfit`, `forestploter`, and `ggswim`) and the pitfalls that trip up first-time converts.

Attendees will leave with a reusable side-by-side translation table and copy-ready SAS and R code for all five figures, together with the simulation programs and ADaM-style example data used to produce every figure in this paper. No prior R experience is assumed — only a working knowledge of ODS Graphics. The goal is not to abandon SAS, but to become bilingual.

INTRODUCTION

If you build figures for oncology or other clinical submissions, you already own a deep toolkit: `PROC SGPLOT`, `PROC SGPANEL`, and the Graph Template Language (GTL) driven through `PROC SGRENDER`. You know how to coax a waterfall plot out of a `VBAR` statement, stack a risk table under a Kaplan-Meier curve with `XAXISTABLE`, and build a two-panel duration-plus-response figure with a GTL `LAYOUT LATTICE`.

More and more, sponsors and CROs are adding R to that workflow. Sometimes it is a validation requirement (an independent programmer reproduces a figure in a second language), sometimes it is a cost or talent decision, and sometimes it is simply where a particular statistical method already lives. Whatever the reason, the SAS programmer is rarely starting from zero. The concepts you use every day in ODS Graphics have close cousins in R's `ggplot2`. This paper is about naming those cousins.

We will do three things. First, establish the single most important mental-model shift: SAS SG procedures are **statement-based**, while `ggplot2` is built on a **grammar of graphics**. Second, provide a translation table you can pin above your desk. Third, walk through five figures — **waterfall, swimmer, Kaplan-Meier, forest, and spider** — and build each one both ways.

The example data. Every figure in this paper is built from simulated CDISC ADaM-style datasets for a fictitious two-arm oncology trial (Drug X vs Control, N=120): ADSL (subject level: `TRT01P`, `TRTDURM`, `STAGEGR1`, `ONGOFL`), ADTR (tumor results: `PARAMCD="SDIAM"`, `AVAL`, `BASE`, `PCHG` by `AVISITN`), ADRS (response: best overall response and response episodes), and ADTTE (progression-free survival: `AVAL`, `CNSR`). The datasets are ADaM-"lite" — core variables only — and entirely simulated; no real subject data appear anywhere. The simulation program, both figure programs, and the data accompany this paper, and all of the SAS and R code shown below has been run as-is against them.

Scope note. This is a knowledge-mapping paper, not an argument that one tool is better. SAS remains an excellent choice for these figures. The aim is to let a fluent SAS programmer read, write, and debug the R equivalents with confidence.

THE ONE IDEA THAT CHANGES EVERYTHING: GRAMMAR OF GRAPHICS

In ODS Graphics you describe a plot as a **sequence of statements inside a procedure**. Each statement (VBAR, SCATTER, REFLINE, SERIES, HIGHLOW) draws one thing, and options after the slash control appearance. The overlay you see is the accumulation of those statements:

```
proc sgplot data=waterfall;
  vbar usubjid / response=bpchg group=bor;
  refline 20 -30 / lineattrs=(pattern=shortdash);
  xaxis display=none discreteorder=data;
  yaxis values=(-100 to 40 by 20);
run;
```

In ggplot2 you describe a plot as **a data set plus a set of layers combined with +**. Every layer is a *geom* (a geometric object: bars, points, lines), fed by *aesthetic mappings* (aes()) that connect data columns to visual channels (x, y, color, fill, group). The same waterfall skeleton:

```
library(ggplot2)
ggplot(wf, aes(x = USUBJID, y = BPCHG, fill = BOR)) +
  geom_col() +
  geom_hline(yintercept = c(20, -30), linetype = "dashed") +
  scale_y_continuous(breaks = seq(-100, 40, 20))
```

Line them up and the correspondence is almost one-to-one: VBAR → geom_col(), GROUP=bor → fill = BOR, REFLINE → geom_hline(), YAXIS VALUES= → scale_y_continuous(). The ranked ordering is the same idea in both languages: sort the data first, then freeze that order (XAXIS DISCRETEORDER=DATA in SAS; locked factor levels in R). Once you see that a **SAS plot statement is a ggplot layer**, the rest is vocabulary.

Two more structural equivalences carry most of the remaining weight:

- **GTL `LAYOUT OVERLAY`** — everything drawn in one set of axes — is simply the default behavior of adding geoms in ggplot2.
- **GTL `LAYOUT LATTICE` and `PROC SGPANEL PANELBY`** — small multiples — become facet_wrap() / facet_grid(), or, when the panels are genuinely different plots (like the waterfall-plus-duration figure), the patchwork package, which stacks independent ggplots with plot1 / plot2.

THE TRANSLATION TABLE: SAS ODS GRAPHICS → R GGPLOT2

What you want to do	SAS ODS Graphics	R / ggplot2 (and friends)
Start a graph	proc sgplot data=; / begingraph;	ggplot(data, aes(...))
Add one plot element	a plot statement (VBAR, SCATTER, SERIES, HIGHLOW)	a geom layer added with +
Overlay in one panel	multiple statements / LAYOUT OVERLAY	multiple geoms added with +
Small multiples	PROC SGPANEL PANELBY / GTL LAYOUT LATTICE	facet_wrap() / facet_grid()
Stack different plots	GTL LAYOUT LATTICE with row weights	patchwork (p1 / p2) or cowplot
Map a variable to color/fill	GROUP=	aes(colour =), aes(fill =)
Group lines by subject	GROUP=usubjid	aes(group = USUBJID)
Bar chart of a response	VBAR x / RESPONSE=y	geom_col() (pre-summarized) / geom_bar()
Horizontal bars / ranges	HIGHLOW ... / TYPE=BAR	geom_segment() or horizontal geom_col()
Points / markers	SCATTER	geom_point()

Connected line / step	SERIES / STEP	geom_line() / geom_step()
Reference line	REFLINE	geom_hline() / geom_vline()
Shaded band (e.g. response zone)	GTL BANDPLOT	geom_rect() / annotate("rect", ...)
Order categories by a value	sort + DISCRETEORDER=DATA / CATEGORYORDER=	arrange() + frozen factor levels / forcats::fct_reorder()
Fixed color/symbol per level	DATTRMAP= attribute map	scale_*_manual(values =) / scale_*_identity()
Custom glyphs (arrows, daggers)	SYMBOLCHAR (Unicode)	Unicode in geom_text() / custom shape scale
Log axis (odds/hazard ratios)	XAXIS TYPE=LOG	scale_x_log10()
At-risk / statistics table	XAXISTABLE / YAXISTABLE	risk-table helpers or a patchwork panel
Axis ranges & ticks	YAXIS VALUES=(...)	scale_y_continuous(breaks=, limits=)
Fonts, sizes, wall, legend look	*ATTRS=, WALLDISPLAY=, KEYLEGEND	theme(), element_text(), guides()
Titles and axis labels	TITLE, LABEL=	labs(title=, x=, y=)
Reusable template	PROC TEMPLATE + SGRENDER	a custom function or a reusable theme_*()
Export the figure	ODS destination	ggsave("fig.png", width=, height=, dpi=)

Table 1. Translation Table: SAS ODS Graphics to R ggplot2

FIGURE 1 — WATERFALL PLOT

Clinical purpose. One ranked bar per subject showing best percent change in tumor size from baseline, colored by best confirmed response, with reference lines at the RECIST thresholds: −30% from baseline marks a response, and +20% marks progression — measured from the *nadir* (the lowest value reached), not from baseline, which is why the two lines are not symmetric around zero. On this baseline-referenced plot the +20% line is therefore an orientation convention rather than the formal PD criterion — a PD-colored bar can legitimately sit below it, showing the subject's best moment before later regrowth off the nadir.

Input data. ADTR supplies PCHG by visit; ADRS (PARAMCD="BOR") supplies the response color. The only derivation is the best (minimum) post-baseline PCHG per subject — and it is the same derivation in both languages.

The SAS way. Derive best percent change, sort descending, and let VBAR do the rest. The DATTRMAP= attribute map pins each response level to a fixed color:

```
proc sql;
  create table waterfall as
  select t.usubjid, min(t.pchg) as bpchg, r.avalc as bor
  from adam.adtr(where=(avisitn > 0 and trt01pn = 1)) as t
  left join adam.adrs(where=(paramcd = "BOR")) as r
    on t.usubjid = r.usubjid
  group by t.usubjid, r.avalc
  order by calculated bpchg desc;
quit;

proc sgplot data=waterfall dattrmap=attrmap;
  vbar usubjid / response=bpchg group=bor attrid=bor barwidth=1;
  refline 20 -30 / lineattrs=(pattern=shortdash);
  xaxis display=none discreteorder=data;
  yaxis label="Change from Baseline (%)" values=(-100 to 40 by 20);
```

```
run;
```

The R way. The identical derivation in `dplyr`, then three layers:

```
library(ggplot2); library(dplyr)

wf <- adtr %>%
  filter(TRT01PN == 1, AVISITN > 0) %>%
  group_by(USUBJID) %>%
  summarise(BPCHG = min(PCHG)) %>%
  left_join(bor, by = "USUBJID") %>%          # BOR from ADRS
  arrange(desc(BPCHG)) %>%
  mutate(USUBJID = factor(USUBJID, levels = USUBJID))  # freeze ranked
order

ggplot(wf, aes(x = USUBJID, y = BPCHG, fill = BOR)) +
  geom_col(width = 1) +
  geom_hline(yintercept = c(20, -30), linetype = "dashed") +
  scale_fill_manual(values = bor_pal) +       # the DATTRMAP analog
  labs(x = NULL, y = "Change from Baseline (%)",
       fill = "Best Overall Response") +
  theme_minimal() +
  theme(axis.text.x = element_blank(), axis.ticks.x = element_blank())
```

Figure 1 shows the R output; the SAS program produces the equivalent figure from the same data.

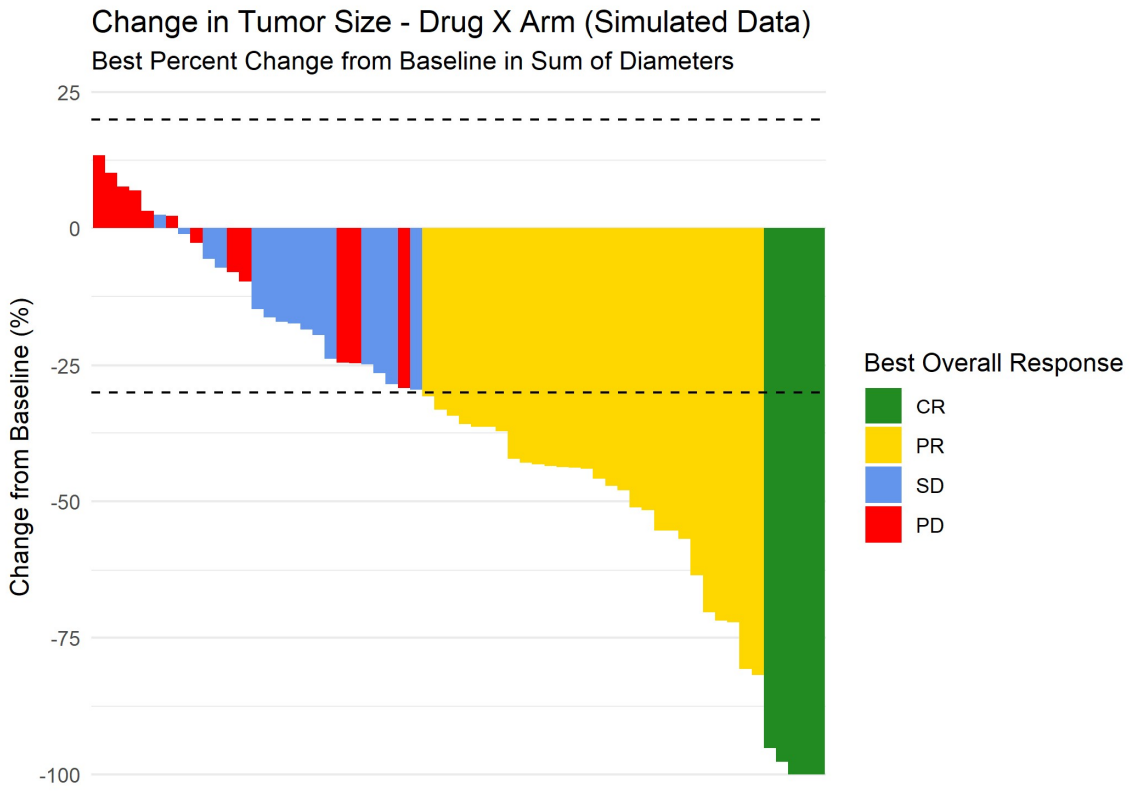


Figure 1. Waterfall Plot of Best Percent Change in Tumor Size (R Output, Simulated Data)

SAS → R notes. The derivation step hides the most common beginner surprise, and it's worth naming the concept behind it: the **factor**, R's type for categorical data, which has no SAS counterpart. Where SAS has only character and numeric, a factor carries its own table of allowed **levels**, and the level order — not the data order — drives axis order, legend order, and color assignment. That's why sorting alone

changes nothing: `arrange(desc(BPCHG))` ranks the rows, and `mutate(USUBJID = factor(USUBJID, levels = USUBJID))` locks that ranking into the levels — together they are the counterpart of sorting plus `DISCRETEORDER=DATA`. Factor levels also exist with zero observations, so an empty category (a response level nobody achieved) can still be listed in a legend when you ask for it (`limits =` in the scale). The closest SAS analogy is a character variable plus a format and `ORDER=DATA` — except the ordering lives in the data itself rather than in the procedure call. Beyond that: `GROUP=` becomes `fill =`, the `DATTRMAP` attribute map becomes `scale_fill_manual(values = )`, and to split by treatment arm, add `facet_wrap(~ TRT01P)` — the direct analog of `PROC SGPANEL PANELBY`.

FIGURE 2 — SWIMMER PLOT

Clinical purpose. One horizontal lane per subject: bar length is time on study, colored by disease stage, with an overlaid response episode, markers for response start/end, and an arrow for ongoing treatment.

Input data. ADSL supplies the lane (`TRTDURM`, `STAGEGR1`, `ONGOFL`); ADRS (`PARAMCD="RSPDUR"`) supplies response episode start/end days, converted to months.

The SAS way. Layer a `HIGHLOW ... TYPE=BAR` for the lane, a second `HIGHLOW` for the response interval, and `SCATTER` statements for the event markers, unified by a `DATTRMAP` so colors stay fixed across statements. `SYMBOLCHAR` turns a Unicode right arrow into a marker for ongoing treatment:

```
proc sgplot data=swim dattrmap=attrmap;
  symbolchar name=arrow char='2192'x;          /* -> = ongoing */
  highlow y=subjid low=zero high=trtdurm / type=bar
          group=stagegr1 attrid=stage barwidth=0.7;
  highlow y=subjid low=rspstartm high=rspdrawm / type=line
          lineattrs=(color=black thickness=3);
  scatter y=subjid x=rspstartm / markerattrs=(symbol=trianglefilled
size=9);
  scatter y=subjid x=rspendm   / markerattrs=(symbol=circlefilled size=9);
  scatter y=subjid x=arrowx    / markerattrs=(symbol=arrow size=14);
  xaxis label="Months on Treatment" values=(0 to 32 by 4);
  yaxis discreteorder=data reverse display=(nolabel);
run;
```

The R way. Each SAS statement becomes one geom; `arrow()` on a short segment replaces `SYMBOLCHAR`:

```
ggplot(swim) +
  geom_segment(aes(y = SUBJID, yend = SUBJID, x = 0, xend = TRTDURM,
                   colour = STAGEGR1), linewidth = 5) +
  geom_segment(data = subset(swim, !is.na(RSPSTARTM)),
               aes(y = SUBJID, yend = SUBJID,
                   x = RSPSTARTM, xend = RSPDRAWM),
               colour = "black", linewidth = 1.2) +
  geom_point(data = subset(swim, !is.na(RSPSTARTM)),
             aes(y = SUBJID, x = RSPSTARTM), shape = 17, size = 2.5) +
  geom_point(data = subset(swim, !is.na(RSPENDM)),
             aes(y = SUBJID, x = RSPENDM), shape = 16, size = 2.5) +
  geom_segment(data = subset(swim, ONGOFL == "Y"),
               aes(y = SUBJID, yend = SUBJID,
                   x = TRTDURM, xend = TRTDURM + 1),
               arrow = arrow(length = unit(0.12, "in"))) +
  scale_colour_manual(values = stage_pal) +
  labs(x = "Months on Treatment", y = "Subjects Received Study Drug",
       colour = "Disease Stage")
```

For a shortcut, the **`ggswim`** package (an actively maintained `ggplot2` extension introduced at R/Medicine 2025) provides `geom_swim_lane()` and `geom_swim_marker()`, handling the fiddly legend composition for you. Figure 2 shows the R output.

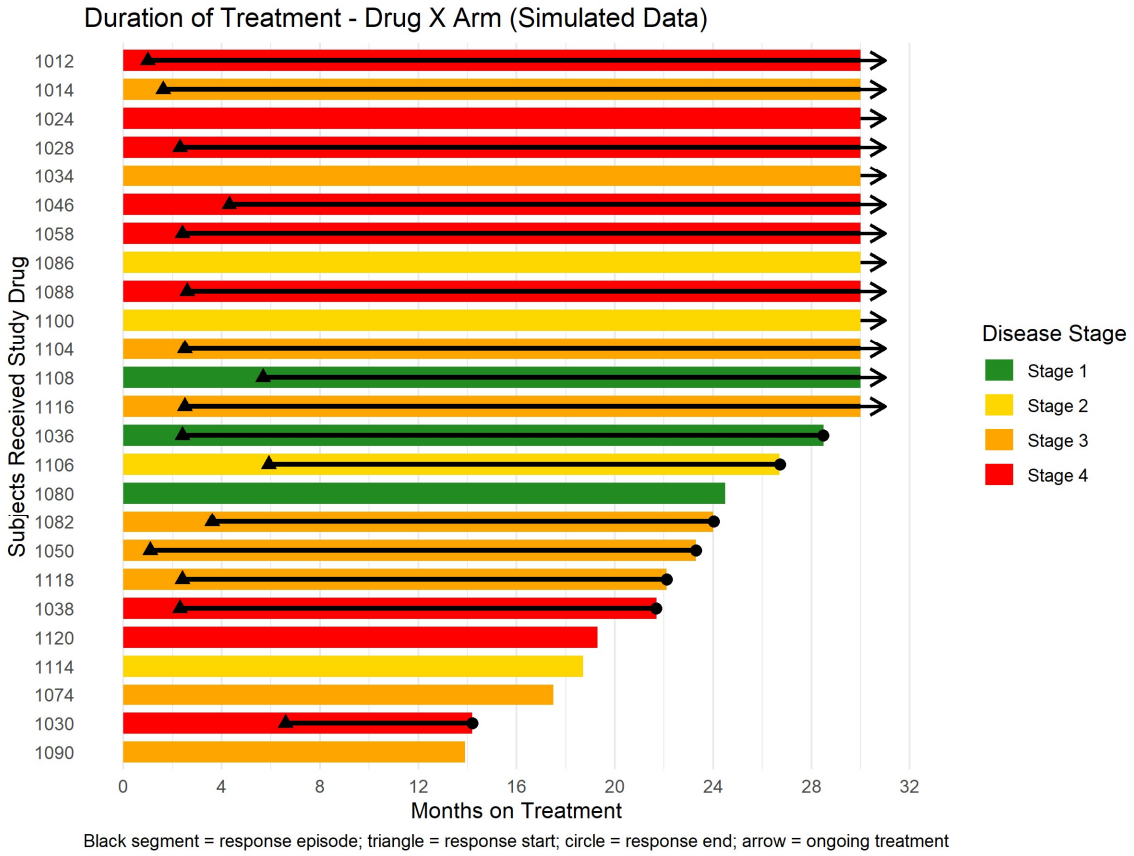


Figure 2. Swimmer Plot of Treatment Duration and Response Episodes (R Output, Simulated Data)

SAS → R notes. `HIGHLOW TYPE=BAR` maps to a thick `geom_segment()` (or a horizontal `geom_col()`). Your `DATTRMAP` becomes `scale_colour_manual(values = c(...))` — this is exactly the "fixed color per level" row of the translation table. `SYMBOLCHAR` arrows become either `arrow()` on a segment or a Unicode glyph in `geom_text()`. Reversing the y-axis (`YAXIS REVERSE`) is reversed factor levels or `scale_y_discrete(limits = rev)`. One caution on axis ranges: this figure's `scale_x_continuous(limits = c(0, 32))` looks like `XAXIS VALUES=(0 TO 32 BY 4)`, but the semantics differ — SAS clips the *view*, while ggplot2 scale limits *remove* out-of-range observations (points become missing, lines break), silently. In a regulated figure that distinction matters. To zoom without dropping data, use `coord_cartesian(xlim = c(0, 32))` instead.

FIGURE 3 — KAPLAN-MEIER PLOT

Clinical purpose. Stepped survival curves by stratum with censoring ticks and a number-at-risk table beneath the plot — the figure reviewers scrutinize most.

Input data. ADTTE (`PARAMCD="PFS"`): `AVAL` in months and the ADaM censoring flag `CNSR` (0 = event, 1 = censored).

The SAS way. `PROC LIFETEST` produces the estimates; capturing its `SurvivalPlot` ODS output lets you redraw the figure in `PROC SGPLOT` with full control — `STEP` for the curves, `SCATTER` for censoring ticks, and `XAXISTABLE` for the at-risk table:

```
proc lifetest data=adam.adtte(where=(paramcd = "PFS"))
```

```
     plots=survival(atrisk=0 to 30 by 6);
  time aval * cnsr(1);
  strata trt01p;
  ods output survivalplot=km;
run;

proc sgplot data=km;
  step x=time y=survival / group=stratum name="s";
  scatter x=time y=censored / markerattrs=(symbol=plus) group=stratum;
  xaxistable atrisk / x=tatrisk class=stratum colorgroup=stratum
            title="Number at Risk" location=outside;
  xaxis label="Months";  yaxis label="Survival Probability";
  keylegend "s" / location=inside position=topright across=1;
run;
```

The R way. The `ggsurvfit` package is the most `ggplot2`-native approach: the result is an ordinary ggplot, so every layer and tweak is added with the same `+` you use everywhere else:

```
library(survival); library(ggsurvfit)
tte <- subset(adtte, PARAMCD == "PFS")

survfit2(Surv(AVAL, CNSR == 0) ~ TRT01P, data = tte) |>
  ggsurvfit(linewidth = 1) +
  add_censor_mark(shape = 3) +
  add_risktable() +
  scale_ggsurvfit()
```

The older `survminer` package builds the same figure through one function call (`ggsurvplot(fit, risk.table = TRUE, ...)`) and remains widely used, but it returns a composite object rather than a plain ggplot, so downstream customization works differently. For a paper about transferring the grammar of graphics, `ggsurvfit` is the better habit to form. Figure 3 shows the `ggsurvfit` output.

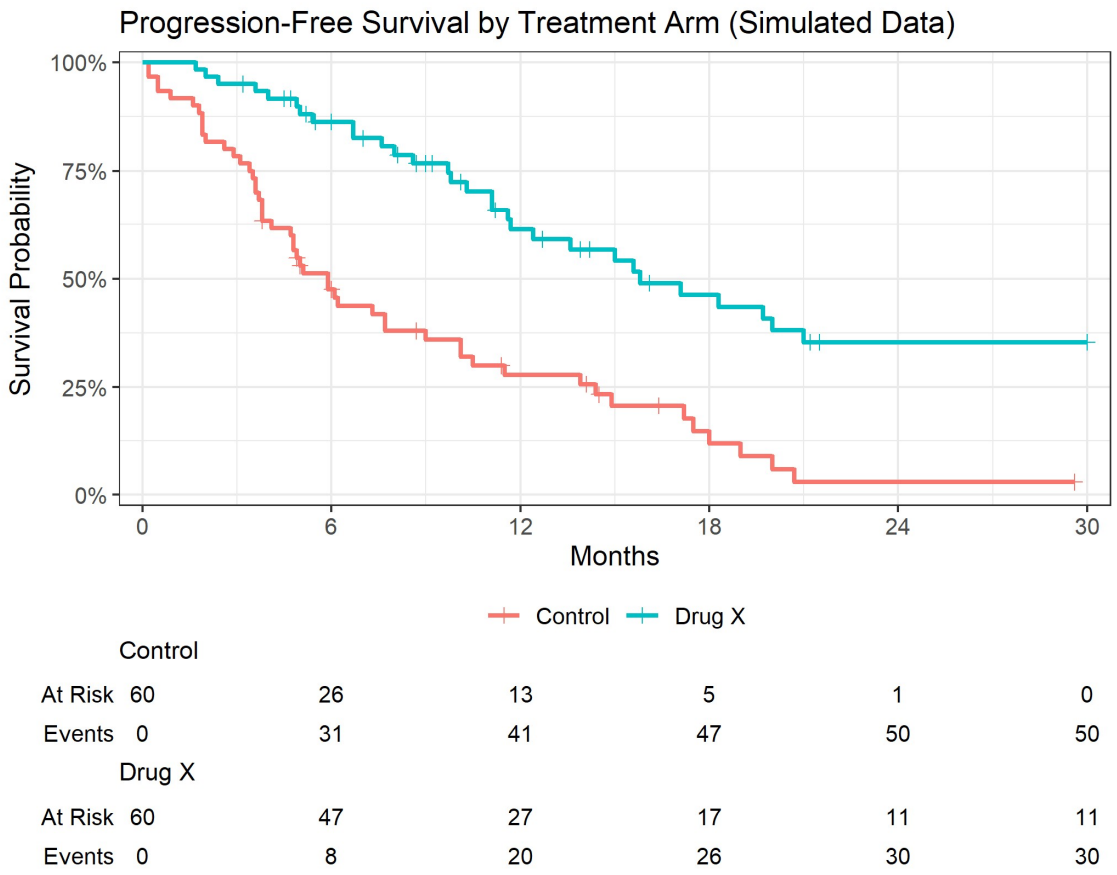


Figure 3. Kaplan-Meier Plot of Progression-Free Survival with At-Risk Table (R Output, Simulated Data)

SAS → R notes. Watch the censoring flip: ADaM and `PROC LIFETEST` describe *censoring* (`TIME aval*cnsr(1)` names the censored value), while R's `Surv()` wants an *event* indicator — hence `CNSR == 0`. Getting this backwards is the single most common KM translation bug, and the curves look plausible either way. Beyond that, R actually *shortens* your code: `add_risktable()` replaces the hand-built `XAXISTABLE`, and the fit replaces the `PROC LIFETEST` + ODS-OUTPUT round trip. `STRATA trt01p` is expressed in the model formula `~ TRT01P`. Note that `survfit2()` is `ggsurvfit`'s drop-in wrapper around `survival::survfit()` — same estimates, but it also stores the calling context so axis and strata labels come out right. `ggsurvfit` slots straight into the grammar of graphics, so you keep customizing with ordinary `+ theme(...)` and `+ labs(...)`.

FIGURE 4 — FOREST PLOT

Clinical purpose. Per-subgroup hazard ratios with confidence intervals on a log scale, a "Favors Treatment / Favors Control" reference line at 1, and aligned numeric columns (N, HR, CI).

Input data. ADTTE again — plus a small analysis step: a Cox model per subgroup. In SAS that is a `PROC PHREG` call per subset (a short macro); in R, a `coxph()` call per subset (a short function). The figure input is the same seven-row summary either way.

The SAS way. `SCATTER` with `XERRORUPPER`/`XERRORLOWER` for the intervals, `XAXIS TYPE=LOG`, `REFLINE` at 1, and `YAXISTABLE` statements to lay the numeric columns beside the plot:

```
proc phreg data=tte;              /* once per subgroup, via a macro */
  where &where;
```

```
  model aval * cnsr(1) = trtind / risklimits;
  ods output parameterestimates=pe;
run;

proc sgplot data=forest noautolegend;
  scatter y=subgroup x=hr / xerrorlower=lcl xerrorupper=ucl
          markerattrs=(symbol=squarefilled size=9);
  refline 1 / axis=x lineattrs=(pattern=shortdash);
  yaxistable n hrci / location=outside position=left;
  xaxis type=log label="Hazard Ratio (Drug X vs Control)"
        values=(0.25 0.5 1 2);
  yaxis discreteorder=data reverse display=(nolabel);
  inset "Favors Drug X" / position=bottomleft;
  inset "Favors Control" / position=bottomright;
run;
```

The R way. The table-alignment work that `YAXISTABLE` does is exactly what dedicated packages exist for. **`forestploter`** treats the whole figure as a table and is the closest match to the SAS layout:

```
library(survival); library(forestploter)

hrsub <- function(d, label) {              # one Cox model per subgroup
  s <- summary(coxph(Surv(AVAL, CNSR == 0) ~ TRTIND, data = d))
  data.frame(Subgroup = label, N = nrow(d),
             HR  = s$conf.int[1, "exp(coef)"],
             LCL = s$conf.int[1, "lower .95"],
             UCL = s$conf.int[1, "upper .95"])
}
dt <- rbind(hrsub(tte, "Overall"),
            hrsub(subset(tte, SEX == "F"), "Female"), ...)

dt$`HR (95% CI)` <- sprintf("%.2f (%.2f, %.2f)", dt$HR, dt$LCL, dt$UCL)
dt$` ` <- paste(rep(" ", 30), collapse = "")  # blank column -> the CI plot

tm <- forest_theme(base_size = 11, ci_pch = 15, ci_lwd = 1.8,
                   refline_lty = "dashed", arrow_cex = 0.85)

p <- forest(dt[, c("Subgroup", "N", " ", "HR (95% CI)")],
            est = dt$HR, lower = dt$LCL, upper = dt$UCL,
            ci_column = 3, ref_line = 1,
            x_trans = "log", ticks_at = c(0.25, 0.5, 1, 2),
            arrow_lab = c("Favors Drug X", "Favors Control"),
            theme = tm)

ggsave("forest.png", p, width = get_wh(p)[1], height = get_wh(p)[2])
```

If you want to stay inside pure `ggplot2`, build three aligned panels (label column, CI plot with `geom_point()` + `geom_errorbarh()` + `scale_x_log10()` + `geom_vline(xintercept = 1)`, numeric column) and glue them with `patchwork`. Other options include `forestplot`, `ggforestplot`, and `forester`. Figure 4 shows the `forestploter` output.

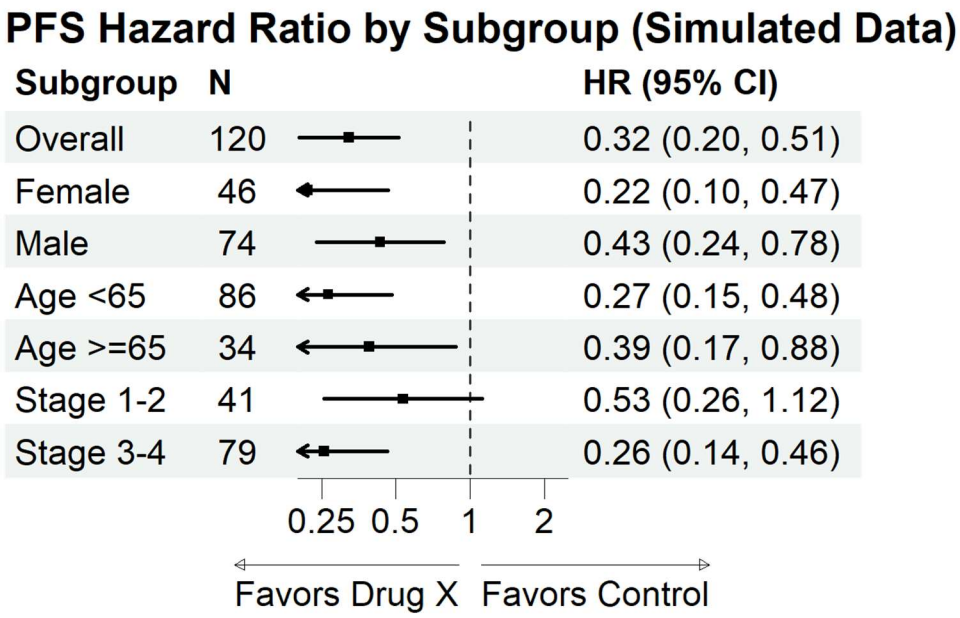


Figure 4. Forest Plot of PFS Hazard Ratios by Subgroup (R Output, Simulated Data)

SAS → R notes. `XERRORUPPER/XERRORLOWER` → `geom_errorbarh(aes(xmin=, xmax=))`. `XAXIS TYPE=LOG` → `scale_x_log10()` or `x_trans = "log"`. `REFLINE 1` → `geom_vline(xintercept = 1)` or `ref_line = 1`. The multiple `YAXISTABLE` statements are the part you *don't* rebuild by hand — that alignment is precisely what `forestploter` automates. And the analysis step translates just as cleanly: a `PHREG` macro loop becomes a small function applied per subgroup.

FIGURE 5 — SPIDER PLOT

Clinical purpose. One connected line per subject tracking percent change from baseline over time (weeks), colored by response category, with a zero reference line and an end-of-line arrow for subjects still on treatment.

Input data. ADTR supplies `PCHG` by `AVISITN` (week); ADRS supplies the response color; ADSL's `ONGOFL` drives the arrow glyph.

The SAS way. A `SERIES` with `GROUP=usubjid` draws one line per subject, and `GROUPLC=` colors the lines by response group rather than by subject — a two-variable trick with a one-aesthetic answer in R. `SYMBOLCHAR` again supplies the Unicode arrow:

```
proc sgplot data=spider dattrmap=attrmap;
  symbolchar name=arrow char='2192'x;
  refline 0 / axis=y lineattrs=(pattern=shortdash);
  series x=week y=pchg / group=usubjid grouplc=bor lcattrid=bor
         lineattrs=(thickness=2);
  scatter x=week y=pchg / group=bor attrid=bor
          markerattrs=(symbol=circlefilled size=6);
  scatter x=arrowx y=arrowy / group=bor attrid=bor
          markerattrs=(symbol=arrow size=12);
  xaxis label="Week" values=(0 to 48 by 6);
  yaxis label="Change From Baseline (%)";
run;
```

The R way.

```
ggplot(sp, aes(x = AVISITN, y = PCHG, group = USUBJID, colour = BOR)) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_line(linewidth = 0.8) +
  geom_point(size = 1.8) +
  geom_text(data = ends,        # last point of ongoing subjects
            aes(x = AVISITN + 1, label = "\u2192"),
            size = 5, show.legend = FALSE) +
  scale_colour_manual(values = bor_pal) +
  labs(x = "Week", y = "Change From Baseline (%)",
       colour = "Best Overall Response")
```

Figure 5 shows the R output.

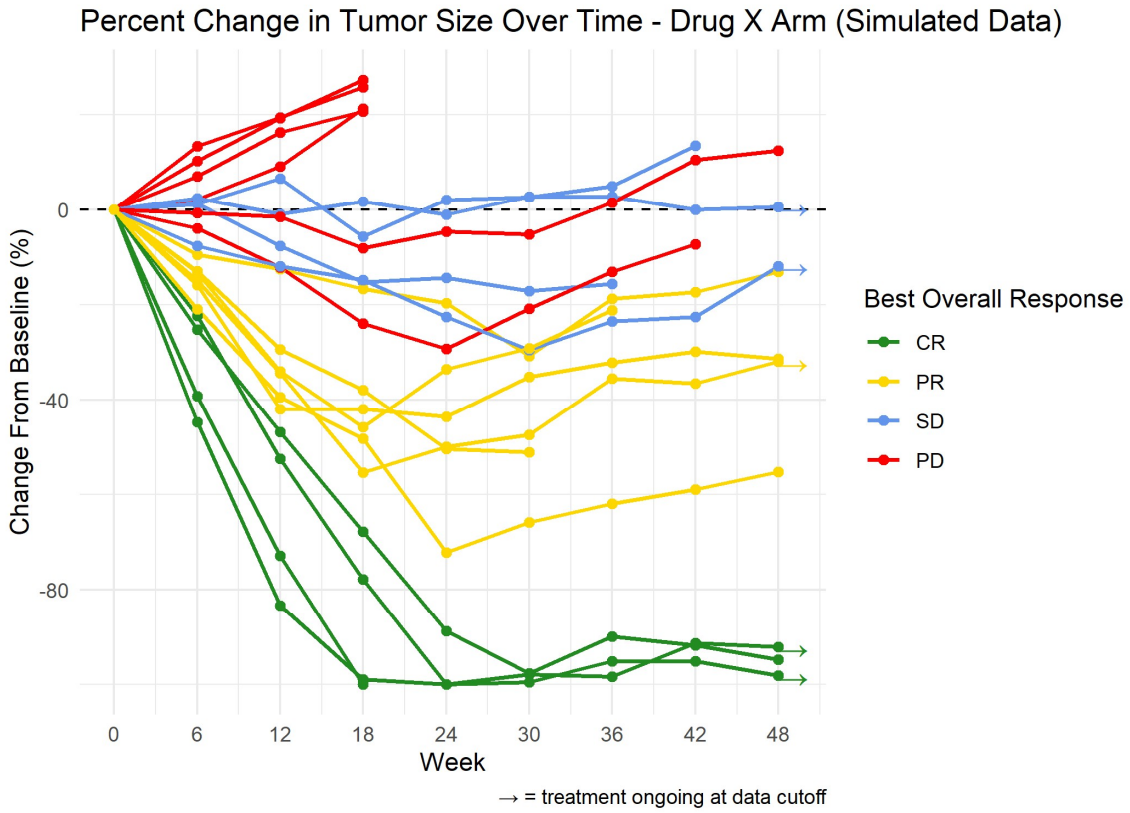


Figure 5. Spider Plot of Percent Change in Tumor Size Over Time (R Output, Simulated Data)

SAS → R notes. This is the cleanest mapping of the five. `SERIES ... GROUP=usubjid GROUPLC=bor` → `geom_line(aes(group = USUBJID, colour = BOR))`: the separate line-color variable collapses into a single `colour =` aesthetic. `SYMBOLCHAR` Unicode glyphs (`'2192'x` →, `'2020'x` †, `'2021'x` ‡) drop straight into a `geom_text(aes(label = ...))` layer.

Notice also that the `geom_text()` layer names no `y=` — it doesn't need one. Mappings in the top-level `ggplot(aes())` are inherited by every layer: a layer's own `aes()` overrides what it names (`x`), adds what's new (`label`), and quietly inherits the rest (`y`, `group`, `colour`). This is the deeper half of the "statement = layer" idea: SGPLOT statements are self-contained, while ggplot layers state only their deviations from a shared contract. Layers can also bring their own data (`data = ends`), which SGPLOT statements cannot — the SAS version of this figure merges the arrow coordinates into the one dataset as extra columns for exactly that reason. When a layer should ignore the contract entirely, `inherit.aes = FALSE` makes it SGPLOT-style self-contained.

A NOTE ON GTL, TEMPLATES, AND MULTI-PANEL FIGURES

The most advanced figure in common oncology decks — waterfall + treatment-duration + baseline, three stacked panels sharing one x-axis — is a GTL `LAYOUT LATTICE` with row weights, rendered via `PROC SGRENDER`. The R analog is **`patchwork`**: build each panel as its own ggplot and combine with `p_duration / p_waterfall / p_baseline + plot_layout(heights = c(0.3, 0.45, 0.25))`. The `PROC TEMPLATE` idea of a reusable, parameterized template becomes an R **function** that takes a data frame and returns a ggplot, or a shared **`theme_*()`** applied across figures — the same "define once, render many" discipline in different clothing.

PRACTICAL CONSIDERATIONS FOR CLINICAL REPORTING

A few things worth flagging for a regulated environment, without overclaiming:

- **Reproducibility & environments.** R package versions matter for byte-identical output. Tools such as `renv` (project-level package snapshots) play the role that a fixed SAS release plays in your validation story.
- **Validation.** Recreating a figure in R is a natural independent check against the SAS original; treat visual and underlying-data equivalence as separate confirmations. The paired programs accompanying this paper — same ADaM inputs, same palette, both languages — are a template for exactly that workflow.
- **ADaM conventions travel well.** Because both programs start from the same ADaM variables (`PCHG`, `CNSR`, `PARAMCD`), the derivations line up one-to-one across languages — but watch convention flips like `CNSR` (censoring flag) versus `Surv()`'s event indicator.
- **The pharmaverse.** An industry-backed R ecosystem for clinical trials (for example `admiral` for ADaM-style data and `ggsurvfit` for survival figures) is converging on exactly these deliverables and is worth citing as the direction of travel.
- **Fonts and export.** `ggsave(..., width, height, dpi)` fixes dimensions the way an ODS destination does; set them explicitly so figures are not silently rescaled.

CONCLUSION

The distance from ODS Graphics to `ggplot2` is shorter than it looks. A SAS plot statement is a ggplot layer; `GROUP=` is an aesthetic; `PANELBY`/`LAYOUT LATTICE` is faceting or `patchwork`; `DATTRMAP` is a manual scale; `XAXISTABLE` is a risk-table helper or an aligned panel. For the two figures with the most bookkeeping — Kaplan-Meier and forest — mature packages (`ggsurvfit`, `forestploter`) actually reduce the code you write. Every figure in this paper was produced, unmodified, by the accompanying SAS and R programs from the same simulated ADaM datasets — proof by construction that the translation table works. The point is not to leave SAS behind, but to be fluent in both.

REFERENCES

CDISC. *Analysis Data Model Implementation Guide (ADaMIG).* Available at https://www.cdisc.org/standards/foundational/adam.

Heath, D. *Creating Oncology Figures Using ODS Graphics.* SAS Institute Inc.

SAS Institute Inc. *SAS/GRAPH and ODS Graphics documentation* (`PROC SGPLOT`, `PROC SGPANEL`, Graph Template Language).

Wickham, H. *ggplot2: Elegant Graphics for Data Analysis.* Springer.

Kassambara, A., Kosinski, M., et al. *survminer: Drawing Survival Curves using ggplot2.* CRAN.

Sjoberg, D. D., et al. *ggsurvfit: Flexible Time-to-Event Figures.* CRAN.

Dayimu, A. *forestploter: Draw Forest Plot.* CRAN.

Hanna, R., et al. *ggswim* (ggplot2 extension for swimmer plots), R/Medicine 2025.

pharmaverse: https://pharmaverse.org

SAS-side example references: support.sas.com/resources/papers/proceedings14/SAS130-2014.pdf; proceedings16/7520-2016.pdf; proceedings17/SAS0675-2017.pdf.

PROGRAMS AND DATA

The complete, tested programs behind this paper — `01_simulate_adam_data.sas` (simulates ADSL, ADTR, ADRS, and ADTTE and exports CSV copies), `02_sas_figures.sas` (all five figures in SAS), and `03_r_figures.R` (all five figures in R) — are available from the authors, together with the simulated datasets. The simulation uses a fixed seed, so every figure in this paper reproduces exactly.

CONTACT INFORMATION

Your comments and questions are valued and encouraged. Contact the authors at:

George Wu — Kardigan — gwu@kardigan.bio
Jaime Yan — Kardigan — jayan@kardigan.bio
Jessie Yan — Kardigan — jyan@kardigan.bio

SAS and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. R is free software under the GNU GPL. Other brand and product names are trademarks of their respective companies.

APPENDIX — FIVE-FIGURE CHEAT SHEET

Figure	Input ADaM	Core SAS statement(s)	Core R layer(s)	Package beyond base ggplot2
Waterfall	ADTR + ADRS	`VBAR / RESPONSE= GROUP=` + `DISCRETEORDER=DATA`	`geom_col()` + frozen factor levels	—
Swimmer	ADSL + ADRS	`HIGHLOW TYPE=BAR` + `SCATTER` + `DATTRMAP`	`geom_segment()` + `geom_point()` + `scale_*_manual()`	`ggswim` (optional)
Kaplan-Meier	ADTTE	`LIFETEST` + `STEP` + `XAXISTABLE ATRISK`	`ggsurvfit()` + `add_risktable()`	`survival`, `ggsurvfit`
Forest	ADTTE (+PHREG)	`SCATTER XERROR*` + `XAXIS TYPE=LOG` + `YAXISTABLE`	`coxph()` + `forest()` (table + CI)	`forestploter`
Spider	ADTR + ADRS	`SERIES GROUP= GROUPLC=` + `SYMBOLCHAR` + `REFLINE 0`	`geom_line(aes(group=))` + `geom_text()`	—

Table 2. Five-Figure Cheat Sheet